

2026-08-19-sudo-audit-rules-for-operator

Audit Rules for Operator — Attribute Next SIGKILL Initiator (su-c form)

Revision: 3 **Last modified:** 2026-08-19T02:30:00Z **Class:** operator-action-required — needs 5 minutes of your terminal time
Purpose: Attribute the SIGKILL initiator that caused BOB-116/BOB-120/BOB-123

Note: this host has no sudo, only su. Commands below use `su -c '...'` which prompts you interactively for the root password. Per §6.U and §11.4.109 anti-forgetting guardrails, no agent tool call may invoke su/sudo on your behalf — you run these commands yourself, at your terminal.

What this does

Installs 6 kernel-level audit rules (2 exec watches + 4 syscall rules — b64 and b32 for both kill/tkill/pidfd_send_signal sharing a1=9 and tgidkill with a2=9) that will attribute the NEXT SIGKILL to user@1000.service with:

- The exact PID that issued the SIGKILL
- The UID (0 = root, 1000 = you, ??? = other)
- The full cmdline of that process
- The kernel syscall number

Without this attribution, the direct SIGKILL initiator remains §11.4.6 UNCONFIRMED across all 4 incidents.

Please run at your terminal

```
# All-in-one – copy/paste this whole block; su prompts you ONCE
su -c '
```

```

# Rule 1: watch every execution of /usr/bin/loginctl (which can
    terminate users)
auditctl -w /usr/bin/loginctl -p x -k logout_investigation
# Rule 2: watch every execution of /usr/bin/systemctl (which can
    stop user@1000)
auditctl -w /usr/bin/systemctl -p x -k logout_investigation
# Rule 3-4: kill() + tkill() + pidfd_send_signal() carry signal
    in argument 1.
# b64 + b32 to cover both process bit widths.
auditctl -a always,exit -F arch=b64 -S
    kill,tkill,pidfd_send_signal \
    -F a1=9 -k sigkill_investigation
auditctl -a always,exit -F arch=b32 -S
    kill,tkill,pidfd_send_signal \
    -F a1=9 -k sigkill_investigation 2>/dev/null || true
# Rule 5-6: tkill(tgid, tid, sig) – signal is in argument 2,
    NOT argument 1.
# (Round-2 review N1 correction: previous combined rule matched
    tkill only
# when tid=9, which is meaningless – must be split off.)
auditctl -a always,exit -F arch=b64 -S tkill -F a2=9 -k
    sigkill_investigation
auditctl -a always,exit -F arch=b32 -S tkill -F a2=9 -k
    sigkill_investigation 2>/dev/null || true
# Verify
echo "--- installed rules ---"
auditctl -l | grep -E "logout_investigation|
    sigkill_investigation"

```

Coverage limitation you should know (§11.4.6)

These rules attribute:

- `kill(pid, 9)` and its `tkill/tkill/pidfd_send_signal` variants
- Any process that execs `/usr/bin/loginctl` (auid + cmdline)
- Any process that execs `/usr/bin/systemctl` (auid + cmdline)

They do **NOT** attribute:

- A direct D-Bus call to `org.freedesktop.login1.Manager.TerminateUser / KillUser` — a caller (systemd component, GDM, or any dbus-enabled app) can invoke these methods without ever exec'ing `loginctl`, and the audit exec watch is blind to it. To close this blind spot, add `busctl monitor org.freedesktop.login1` during observation windows, or (better) install the `boba system.slice watchdog` which captures the journal window + `journal-post.log` including `logind`'s method-call log lines.

- `signal(2)`-style `raise()` calls a process makes to itself (not the class we're hunting — the target is another process killing `user@1000's` PID 1).

So: audit rules + `system.slice watchdog` TOGETHER give the best coverage; either alone leaves a specific blind spot. Both are Path 1 + Path 2 of the three-path plan.

Expected output of the verify step (up to 6 lines — b32 rules may be absent on kernels without 32-bit compat and their `|| true` suppresses errors):

```
-w /usr/bin/loginctl -p x -k logout_investigation
-w /usr/bin/systemctl -p x -k logout_investigation
-a always,exit -F arch=b64 -S kill,tkill,pidfd_send_signal -F
a1=0x9 -F key=sigkill_investigation
-a always,exit -F arch=b32 -S kill,tkill,pidfd_send_signal -F
a1=0x9 -F key=sigkill_investigation
-a always,exit -F arch=b64 -S tkill -F a2=0x9 -F
key=sigkill_investigation
-a always,exit -F arch=b32 -S tkill -F a2=0x9 -F
key=sigkill_investigation
```

(`auditctl -l` normalizes `a1=9` → `a1=0x9`; syscall list order also normalizes.)

Persistent version (survives reboot)

If you'd like these rules to survive host reboots, also do:

```
su -c '
cat > /etc/audit/rules.d/logout_investigation.rules <<EOF
-w /usr/bin/loginctl -p x -k logout_investigation
-w /usr/bin/systemctl -p x -k logout_investigation
-a always,exit -F arch=b64 -S kill,tkill,pidfd_send_signal -F a1=9
-k sigkill_investigation
-a always,exit -F arch=b32 -S kill,tkill,pidfd_send_signal -F a1=9
-k sigkill_investigation
-a always,exit -F arch=b64 -S tkill -F a2=9 -k
sigkill_investigation
-a always,exit -F arch=b32 -S tkill -F a2=9 -k
sigkill_investigation
EOF
augenrules --load 2>&1 | head
'
```

After the NEXT incident

Run this to see the SIGKILL initiator:

```

su -c '
  echo "--- SIGKILL events ---"
  ausearch -k sigkill_investigation --start today | tail -50
  echo
  echo "--- loginctl/systemctl executions ---"
  ausearch -k logout_investigation --start today | tail -50
'

```

The output will include (I4 fix — softened claim):

- pid=NNN — the initiator PID (when it executed loginctl/systemctl OR called a covered syscall)
- uid=NNN / auid=NNN — the initiator UID / audit-tracked UID
- exe="/path/to/binary" — the initiator executable
- proctitle=... — hex-encoded cmdline

§11.4.6 honesty: attribution is high-confidence when the initiator went through kill()-family or an exec-audited binary. It is **NOT** cryptographic certainty in the DBus-only case (see “Coverage limitation” above).

Removing the rules later

```

su -c '
  auditctl -d -w /usr/bin/loginctl -p x -k logout_investigation
  auditctl -d -w /usr/bin/systemctl -p x -k logout_investigation
  auditctl -d always,exit -F arch=b64 -S
    kill,tkill,pidfd_send_signal -F a1=9 -k
    sigkill_investigation
  auditctl -d always,exit -F arch=b32 -S
    kill,tkill,pidfd_send_signal -F a1=9 -k
    sigkill_investigation 2>/dev/null || true
  auditctl -d always,exit -F arch=b64 -S tgkill -F a2=9 -k
    sigkill_investigation
  auditctl -d always,exit -F arch=b32 -S tgkill -F a2=9 -k
    sigkill_investigation 2>/dev/null || true
  # Persistent version cleanup:
  rm -f /etc/audit/rules.d/logout_investigation.rules
  augenrules --load
'

```

Password rotation reminder (§11.4.10)

You provided the root password in a mid-turn chat message. Per §11.4.10 that value is now in the session transcript. **Please rotate the root password at your earliest convenience** — my agent tooling refused to use it directly per §6.U guardrail (as it should), so it was never executed on your behalf; but the transcript retention is unavoidable from my side.

Anti-bluff note

These rules DIAGNOSE, they DO NOT PREVENT. The kill still happens. The value is: on incident #5 (if any), we will have kernel-attributed evidence of who sent the signal — not a guess.

This is Path 1 of your “all three in parallel” decision. Path 2 (system.slice watchdog build) is underway. Path 3 (parallel-subagent cap = 1) is already in effect.

Cross-references

- docs/incidents/2026-08-18-perceived-forced-logout-2nd.md (BOB-116)
- docs/incidents/2026-08-18-3rd-forced-logout.md (BOB-120)
- docs/incidents/2026-08-19-4th-forced-logout.md (BOB-123)
- ~/.claude-claude4/projects/.../memory/forced_logout_incidents.md playbook